Two-Tier System: Complete User Manual

This manual explains, in plain language and one small step at a time, how to build and use the whole project from nothing: a private banking assistant that answers most questions itself, on your own computer, and asks a paid AI (Claude) for help only with the questions its own answer is predicted not to be good enough for. It also shows how to read the results, how to train the system's router, how to train your own versions of the small models, and how to fix the problems people most often meet.

You do not need to know how to code. Every step tells you exactly what to type or click, and what you should see afterwards so you know it worked.

How to use this manual. Work through sections 0 to 9 in order, without skipping, because each depends on the one before. After section 9 you have a working system, and sections 10 to 20 are for reference: read the ones you need.

Contents

0. Words this manual uses
1. What you will end up with
2. Before you start
3. Install the basic tools
4. Get the project files onto your computer
5. Set up the project's Python environment
6. Get a Claude API key
7. Put the two small AI models in Ollama
8. The practice questions
9. Run the system for the first time
10. Reading a run's results
11. The router: how it decides, and how to train it
12. The dashboard
13. Quality scoring, the cache and the maintenance scripts
14. Train your own models on Google Colab (optional)
15. Bring your trained models into Ollama
16. Research scripts (optional)
17. Everyday commands
18. Tests
19. If something goes wrong
20. Appendices

---

0. Words this manual uses

Word	What it means
Terminal (Windows: "PowerShell")	A window where you type commands instead of clicking buttons. It is built into your computer; you do not download it
Command	A line of text you type into the terminal and run by pressing Enter. Commands are shown in a box like this: `like this`
Folder / directory	A place on your computer where files are kept
Repository ("repo")	A folder whose history is tracked by a tool called Git, so changes can be saved, shared and undone. The whole project is one repository
GitHub	A website that stores a copy of a repository online
Clone	Downloading a full copy of a repository onto your computer
Virtual environment (venv)	A private box for this one project's Python software, so it never clashes with anything else on your computer
API key	A long secret code that lets the program use Claude over the internet. It is tied to a billing account, so it must stay private
Model	The AI "brain" that reads a question and writes an answer. This project uses small ones that run on your computer (Phi-3-mini and Qwen2.5) and a larger one, Claude, that runs on Anthropic's computers
Ollama	A free program that runs the small models on your computer
Tier 1 / Tier 2	Tier 1 is the small local model that answers first. Tier 2 is Claude, which answers only escalated questions

Business unit	A part of the bank that has its own small model. There are two: **payments** and **retail bank**
Confidence	How sure the small model sounded of its own wording, from 0 to 1. It is *not* a measure of whether the answer is right
Quality mark	A score from 1 to 5 that Claude, acting as an examiner, gives an answer on five points: correctness, completeness, tone, safety and clarity
Router	The part that decides, for each question, whether to keep the small model's answer or escalate. Here it predicts the answer's quality and escalates if the prediction is too low
Cut-point	The line the router compares its predicted quality with. Below it, the question is escalated
Escalate	To hand a question to something better (and costlier). In this project: first the cache, then Claude
Cache	A store of earlier Claude answers. If a new question is a near-exact repeat of an old one, the old answer is re-used for free
Decision	What finally happened to a question: `LOCAL` (the small model's answer was sent), `CACHE` (an old Claude answer was re-served) or `ESCALATE` (Claude wrote the answer)
Intent	The specific kind of request in a question, such as "block a card". There are 50. The router is always tested on intents it did not see while learning
Synthetic	Made up by a program. All the project's test questions are synthetic, so no real customer data is involved
Run	One go of the pipeline on a batch of questions. Each run has an id and is added to the history
History / log	The CSV files in `results/logs/` that record every question of every run
Fine-tuning	Teaching a small model to do better at banking questions by showing it thousands of examples
Colab	A Google website that lends you a powerful computer with a graphics card (GPU) for a few hours. Fine-tuning needs one
GGUF	A single file that holds a fine-tuned model in a form Ollama can load
Dashboard	A web page that shows charts and tables of how the system is doing

If a word appears that is not here, the manual explains it where it first appears.

---

1. What you will end up with

By the end of section 9, on your own computer, you will have:

1. A folder called `two-tier-system` with all the project's code.
2. Two small models, one for payments questions and one for retail-banking questions, running privately through Ollama.
3. A connection to Claude for the questions that need it.
4. A trained router that decides which answers to keep and which to escalate.
5. A cache that re-serves old Claude answers to repeated questions.
6. A system that records every decision, time, cost and quality mark of every question.
7. A dashboard that shows all of that in charts and tables.

Optionally, you can train your own improved versions of the two small models (sections 14 and 15) and re-train the router (section 11). Everything runs on your computer except the optional training (which borrows Google's computer) and the escalated questions (which go to Claude).

---

2. Before you start

What you need

Item	Details
A computer	Windows, Mac or Linux. At least 8 GB of memory (16 GB is comfortable) and about 15 GB of free disk space. A graphics card is not needed, but makes answers faster
An internet connection	For downloads, and for Claude
Time	Sections 3 to 9: one to two hours, mostly waiting for downloads. That is normal, not a mistake
An Anthropic account	To create a Claude key (section 6). Pay-as-you-go
Optional: a Google account	Only for training your own models (section 14)

What it costs. Everything is free except Claude. A Claude answer to an escalated question cost about $0.0014 in the project's logs, and each quality mark about 0.0012.*A*100 − *questionrunwitheverythingonthereforecostsroughly* * *0.15 to

$0.25**, almost all of it quality marking. Turning marking off (`--noquality`, section 9) brings it to a few cents. These are estimates, and prices change.

Two ways to get your small models: choose now.

- **Path A: base models (recommended first).** Download two ready-made models (section 7). About 30 minutes, no training. The system works fully, though its answers and confidence differ from the fine-tuned models the project was measured with.
- **Path B: fine-tuned version 2 models.** Train them on Google Colab (sections 14 and 15), about 3.7 and 5.5 hours of waiting, or ask the project owner for the finished model files.

You can start with Path A and move to Path B later. This manual's commands work with both; the difference is one edit in section 7.

---

3. Install the basic tools

You need three programs before anything else: **Ollama** (runs the small models), **Python** (runs the project's code) and **Git** (downloads the project). Install them in this order.

3.1 Install Ollama

Windows

1. Go to **ollama.com/download** and click **Download for Windows**.
2. Open the downloaded file (`OllamaSetup.exe`), click **Install**, and wait. You do not need to change any options.
3. Ollama now runs quietly in the background. A small llama icon appears near the clock.

Mac

1. Go to **ollama.com/download** and click **Download for macOS**.
2. Open the downloaded file, drag Ollama into your Applications folder, then open it. Click **Open** if a security warning appears.

Check it worked. Open a terminal (Windows: Start menu, type `PowerShell`, press Enter. Mac: `Cmd + Space`, type `Terminal`, press Enter) and type:

```
ollama --version
```

You should see a version number. **This project needs version 0.12.11 or newer**, because older versions do not return the word probabilities that confidence is computed from. If you see "not recognized" or "command not found", close the terminal, open a new one, and try again.

3.2 Install Python

Windows

1. Go to **python.org/downloads** and click the big **Download Python** button.
2. Open the file. **On the very first screen, tick "Add python.exe to PATH".** This is easy to miss and causes problems later if skipped.
3. Click **Install Now**.

Mac. Go to python.org/downloads, download the macOS installer, open it, and follow the prompts.

Check it worked:

```
python --version
```

You should see `Python 3.11` or newer (the project was tested on 3.12 and 3.14). On a Mac you may need `python3 --version`; if so, use `python3` in place of `python` in the commands below, until section 5 creates the environment, after which `python` works.

3.3 Install Git

Windows. Go to **git-scm.com/downloads**, click **Windows**, open the file, and click **Next** on every screen, then **Install**.
Mac. Type `git --version` in the terminal. If Git is missing, a window offers to install "Command Line Developer Tools": click **Install**.

Check it worked: `git --version` shows a version number.

3.4 Node.js (optional)

Only if you want to change how the dashboard looks (section 12). Download from nodejs.org (version 18 or newer). You do not need it to use the dashboard.

---

4. Get the project files onto your computer

4.1 Download ("clone") the project

1. Open your terminal and move to where you want the project folder to live, for example your Desktop:

   ```
   cd Desktop
   ```

2. Download the project:

   ```
   git clone https://github.com/bipul-ranjan/two-tier-system.git
   ```

3. **You should see** text scroll by and end with a line such as `Resolving deltas: 100% done`.

4. Move into the new folder:

   ```
   cd two-tier-system
   ```

Keep the terminal in this folder for every command in this manual. The folder contains `src`, `scripts`, `data` and other folders; type `dir` (Windows) or `ls` (Mac) to check you can see them. If you are using your own copy of the repository, replace the address with yours.

4.2 If the repository is private, or you want to save changes back to GitHub

GitHub does not accept your normal password in a terminal. Instead you create a **token**:

1. On github.com, click your picture (top right), then **Settings**.
2. Scroll the left menu to the bottom and click **Developer settings**, then **Personal access tokens**, then **Tokens (classic)**.
3. Click **Generate new token (classic)**, give it a name such as `my-laptop`, tick the box beside **repo**, and click **Generate token**.
4. **Copy the long code (it starts `ghp_`) immediately.** You cannot see it again; if you lose it, make a new one.
5. Whenever Git asks for a **username**, type your GitHub username; whenever it asks for a **password**, paste the token.

Your terminal usually remembers it after the first time.

---

5. Set up the project's Python environment

This creates a private box for the project's software, so it does not interfere with anything else.

1. In the project folder, create the environment:

   ```
   python -m venv venv
   ```

   This takes a few seconds and makes a folder called `venv`. You never need to look inside it.

2. Turn it on (this is called "activating" it):

   - **Windows (PowerShell):**

     ```
     Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
     .\venv\Scripts\Activate.ps1
     ```

     (The first line is a one-time permission fix for this window; answer `Y` if asked.)

- **Mac / Linux:**

```
source venv/bin/activate
```

3. **You should see** `(venv)` at the very start of the terminal line. **From now on, every time you open a new terminal to work on this project, do step 2 again first**, or the project's software will not be found.
4. Install the project's packages:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
pip install -r dashboard/backend/requirements.txt
```

   The first `pip install` is large (several GB, because it includes PyTorch for the cache) and takes several minutes. Scrolling text is normal.

Check it worked. Run the tests, which need no models and no key:

```
python -m pytest tests/ -q
```

You should see `158 passed`. If you see `No module named pytest`, the environment is not active (step 3).

---

6. Get a Claude API key

Escalated questions and quality marking use Claude, so the project needs your own private key.

1. Go to **console.anthropic.com** (this is different from claude.ai; the "console" address is for developers). Sign up or log in.
2. Click **Settings**, then **Billing**, and add a payment card. It is pay-as-you-go; normal use costs cents.
3. Click **API Keys** in the left menu, then **Create Key**, name it `two-tier-system`, and click **Create**.
4. **Copy the key immediately.** It starts with `sk-ant-` and is never shown again.
5. Save it on your computer so the project can use it, without putting it in any project file:

   - **Windows (PowerShell):**

```
setx ANTHROPIC_API_KEY "paste-your-real-key-here"
```

     Close the terminal completely and open a new one, then activate the environment again (section 5, step 2). This setting only takes effect in new windows.

   - **Mac / Linux:**

```
echo 'export ANTHROPIC_API_KEY="paste-your-real-key-here"' >> ~/.zshrc
source ~/.zshrc
```

Check it worked. Windows: `echo $env:ANTHROPIC_API_KEY`. Mac: `echo $ANTHROPIC_API_KEY`. It should print your key. If it prints nothing, repeat step 5.

Keep the key private. Never paste it into a chat, an email, or any file you share or push to GitHub: anyone who has it can spend money on your account. If you ever think it has leaked, delete it in the console and make a new one.

---

7. Put the two small AI models in Ollama

The project's settings (`src/config.py`) say which model each business unit uses. As shipped they name the **fine-tuned version 2** models, `payment-assistant-v2` and `retail-bank-assistant-v2`. Those can not be downloaded: they are built from files made by training (sections 14 and 15). So do one of these:

Path A: use the base models (about 30 minutes)

1. Make sure Ollama is running (the llama icon near the clock) and the environment is active. Then, one at a time, waiting for each to finish:

```
ollama pull phi3:mini
```

```
ollama pull qwen2.5:1.5b
```

Each shows a progress bar and downloads a few gigabytes.

2. **Check:** `ollama list` shows both `phi3:mini` and `qwen2.5:1.5b`.
3. Open the file `src/config.py` in a plain text editor (Notepad on Windows, TextEdit on Mac in plain-text mode). Find this part:

```
        "payments": {
            "model": "payment-assistant-v2",
            ...
        "retail_bank": {
            "model": "retail-bank-assistant-v2",
```

and change the two model names to `"phi3:mini"` and `"qwen2.5:1.5b"`. Save the file with the same name, and take care that no `.txt` is added to it.

4. **Check:** run `python -m src.config`. It prints each unit with the model it will use.

Because the base models answer differently from the fine-tuned ones, their confidence is different too, so the router should be retrained (section 11.3) after you have some history.

Path B: use the fine-tuned version 2 models

Train them (section 14) and import them (section 15), or get the two `.gguf` files from the project owner and do section 15. Then `ollama list` shows `payment-assistant-v2` and `retail-bank-assistant-v2`, and `src/config.py` needs no edit.

Check a model directly

```
ollama run phi3:mini "Why was my card payment declined?"
```

(Use whichever model name you now have.) You should see a written answer. Type `/bye` to leave.

---

8. The practice questions

The system is tested on made-up banking questions, so no real customer data is ever involved. **They are already in the project**, in `data/synthetic/`: ten files of 1,600 questions each (16,000 in all). Check:

- **Windows:** `dir data\synthetic` **Mac:** `ls data/synthetic`

You should see `synthetic_bitext_01.csv` to `synthetic_bitext_10.csv` and `synthetic_manifest.csv`. You do not need to do anything else.

Optional commands, only for training models or repeating the project's earliest experiments:

Command	What it does
`python scripts/generate_synthetic_bitext.py`	Re-creates the ten files exactly (it always makes the same ones unless you change `SEED` in the script)
`python scripts/fetch_bitext_banking.py`	Downloads the public Bitext banking dataset and sorts it into payments and retail-bank training files, in `data/raw/`
`python scripts/generate_exception_training_data.py`	Makes synthetic training examples for fraud, hardship and similar scenarios (retail model)
`python scripts/fetch_data.py`, then `python scripts/convert_finqa_to_csv.py`	Downloads two older public datasets (Banking77 and FinQA). Not needed to run the system

`data/README.md` explains each file.

---

9. Run the system for the first time

This sends a batch of practice questions through the whole system: the small models answer first, the router decides which answers to keep, and the rest go to the cache or to Claude.

With `(venv)` showing and Ollama running, type:

```
python -m src.pipeline 10
```

The `10` means "use 10 random practice questions", good for a first try. When it works, try `python -m src.pipeline 100`.

What you should see

1. A line saying `Router: LEARNED (default)` (and a short explanation), then the models loading.
2. One block of text per question, like this:

   ```
   Business Unit  : retail_bank (Retail Bank Assistant v2)
   Scenario       : ACCOUNT / close_account
   Query          : Please help: I want to close my current account. ...
   Tier 1 Answer  : I understand that you would like to close your current account. ...
   Confidence AVG : 0.6770
   Router         : text-probe-05Oct26-1102 | predicted quality 3.24 (cut-point 3.27)
   Decision       : ESCALATE
   Latency        : Tier1 3928 ms ... | Tier2 4631 ms ... | Total 8578 ms
   ```

   Read it as: the answer's predicted quality (3.24) was below the cut-point (3.27), so it was escalated. If a similar question had been answered before, you would see a `Cache hit` line and decision `CACHE`; if the predicted quality were above the cut-point, the decision would be `LOCAL`.
3. A message that the models are being unloaded, then a stage "Scoring answer quality ... (Claude-as-judge)" with progress counts, then a second stage for the discarded Tier 1 drafts.
4. A summary: how many of each decision, median times, and a line such as `Router ...: escalated 31% of questions (payments 23%, retail_bank 38%)`.

Time. Each local answer takes a few seconds (3 to 8 on the author's machine); escalated ones add the time Claude takes. Ten questions take a few minutes including marking.

How to know it worked. Look in the results folder: Windows `dir results\logs`, Mac `ls results/logs`. You should see `results_history.csv`, `results_log_combined.csv` and one file per business unit.

The options

```
python -m src.pipeline [N] [--threshold-router] [--nocache] [--noquality] [--no-redact] [--questions PATH] [--fresh]
[--pool] [--generator-model NAME] [--label-model NAME] [--label-method NAME] [--seed N] [--score-quality-local MODEL]
```

Option	What it does	When to use it
`N` (a number)	How many random questions. Default 100	Always; start small
`--threshold-router`	Use the older rule (escalate when confidence is below 0.71 for payments or 0.60 for retail bank) instead of the learned router	To compare the two routers, or if you have not got a trained router yet
`--nocache`	Do not look for a re-usable old Claude answer	To measure what the cache saves
`--noquality`	Skip the quality marking after the run	To save money while testing. Fill the marks in later (section 13)
`--no-redact`	Turn the personal-data filter off, so raw question text is sent to Claude and written to the log	Only to reproduce an older run. Leave it on otherwise
`--questions PATH`	Draw the questions from a CSV or a folder of CSVs instead of the built-in pool, for example a batch of fresh questions	For fresh wording (see below)
`--fresh`	First make fresh customer questions with a separate local model, replacing the previous fresh batch, then run as usual	When you want different wording every run
`--pool`	Use the built-in pool even if fresh questions have been made the default	To switch fresh questions off for one run
`--generator-model NAME`	The model that writes the fresh questions (only with `--fresh`)	To try a different generator
`--label-`	A separate model that checks each fresh question's label (only with `--fresh`), so the	

`model NAME`	generator does not mark its own work	
`--label-method NAME`	How the judge is asked: `examples` (default), `number`, `shuffled`, `name`, `two-step` or `category`. Measure them first with `scripts/evaluate_label_judge.py` (only with `--fresh`)	
`--seed N`	Draw the same questions every time you pass the same number	To repeat a run exactly
`--score-quality-local MODEL`	Also mark with a local Ollama model, into separate columns	A cheap extra opinion; not the numbers to report

Examples:

```
python -m src.pipeline                          # 100 questions, everything on
python -m src.pipeline 50 --noquality           # a cheap test
python -m src.pipeline 100 --threshold-router   # the old router, for comparison
python scripts/run_and_score.py 100             # the pipeline, then both quality fills as a safety net
```

If you mistype an option, the program stops and lists the valid ones.

Things to know about a run

- **Fresh questions every run.** The built-in pool of 16,000 questions repeats across runs, which flatters the cache. For a different set every time, download a general model once (`ollama pull qwen2.5:7b`) and run `python -m src.pipeline 100 --fresh`. Before anything else it asks that separate model for 100 new questions (a few minutes), checks them for repeats and for the right request type, deletes the previous fresh batch, and then runs exactly as usual, one question at a time. Only the pipeline's own batch is deleted; batches you made with `scripts/generate_questions.py` are kept. The generator is not one of your fine-tuned helpers. Its questions are still made up, and a small model checks their labels, so read a few (`python scripts/generate_questions.py --n 20 --show 20`) before trusting a run.
- **A personal-data filter runs by default.** Your own computer (Tier 1) sees each question as typed, but before anything goes to Claude, the cache or the log, patterned identifiers such as card numbers, emails, phone numbers and account numbers are replaced with placeholders like `[CARD_NUMBER]`. It does **not** catch names or street addresses written in ordinary sentences, and it has not been tested on real customer text, so do not use real customer data. Measure it with `python scripts/evaluate_privacy_filter.py`, and clean older logs with `python scripts/redact_history.py` (a dry run unless you add `--apply`).
- **Do not open `results_history.csv` in Excel during a run.** The program locks it from the start; an open copy blocks the final save. If that happens, close it and run `python scripts/recover_missing_run.py`.
- **The questions are random** each time. To draw the same questions every run (needed for fair comparisons), open `src/pipeline.py`, find `SEED = None` near the top, and change it to a number such as `SEED = 42`.
- **If the program stops at the start with a message about the router,** the trained router is missing or from another scikit-learn version. Section 11 explains the fix; to run once without it, add `--threshold-router`.
- **One question failing** (for example a model that hangs) is skipped with a warning, and all the others are still saved.

---

10. Reading a run's results

Every question of every run is saved in `results/logs/`:

File	Contains
`results_history.csv`	**Every run**, added to each time. The dashboard and the router read this one
`results_log_combined.csv`	The **latest run only**
`results_log_payments.csv`, `results_log_retail_bank.csv`	The latest run only, one file per business unit

Open them in Excel or any spreadsheet program (but not during a run). The **decision** column tells you what happened to each question:

Decision	Meaning	Did Claude get called?
`LOCAL`	The router kept the small model's answer, and it was sent as written	No
`CACHE`	The router escalated, but a near-identical earlier question had a Claude answer, which was re-served	No
`ESCALATE`	The router escalated and Claude wrote the answer	Yes

The most useful columns are `query`, `tier1_answer` (the small model's answer, kept even when discarded), `final_answer` (what the customer got), `decision`, `tier1_confidence_avg`, `router_predicted_quality` and `router_cutoff`, `estimated_cost_usd`,

and the quality marks `quality_overall` (the final answer) and `quality_draft_overall` (the small model's own answer). **`results/logs/README.md` explains all 51 columns.**

A run as one plain table

The text the pipeline prints is hard to read or share. To see a run as one row per question, with a summary and a list of problems:

```
python scripts/export_run_report.py                     # the newest run
python scripts/export_run_report.py --list              # which runs exist
python scripts/export_run_report.py --run "run-18-Oct-26-09:05 AM"
```

It writes a CSV and, if you have run `pip install openpyxl`, a colour-coded Excel file to `results/reports/`. If a run showed `row failed and will be skipped` warnings, those rows were **not saved** (the log silently leaves them out), so save the console text into a file and run `python scripts/export_run_report.py --console run_console.txt` to bring them back and see the true numbers.

Quick summary tables

```
python -m src.evaluate
```

writes two small files to `results/tables/` for the **latest run**: the share resolved locally, average times, the Claude cost, and what sending everything to Claude would have cost. The dashboard (section 12) covers every run and the cache, and is usually the easier way to look.

Looking at the history with Python

```
python
>>> import pandas as pd
>>> h = pd.read_csv("results/logs/results_history.csv")
>>> h["decision"].value_counts()
>>> h.groupby(["business_unit", "decision"]).size().unstack()
>>> exit()
```

---

11. The router: how it decides, and how to train it

11.1 The idea in plain words

Early on, the project decided by asking: "how sure did the small model sound?" That turned out to be a weak guide: a model can sound sure and be wrong, or sound unsure and be fine. So the router now does something different. It looks at **the question, the small model's answer, the confidence and the business unit**, and **predicts the quality mark Claude would give that answer** (1 to 5). If the prediction is below a **cut-point** (about 3.27), the answer is not trusted and the question is escalated.

The router is a simple text model that counts which words and phrases tend to go with good and with poor answers, learned from your own history. In tests on question types it had never seen, it predicted quality about twice as well as confidence did (a correlation of +0.477 against +0.212), and fancier language-model routers did no better.

Because it predicts lower quality for the retail-bank model's answers (they really are weaker), one cut-point sends more retail-bank than payments questions up: in tests, about 45% against 14%.

11.2 Is a trained router already there?

The project includes one, in `models/router/`. It works only if the scikit-learn version on your computer matches the one that trained it. Check:

```
python -c "import json; print(json.load(open('models/router/text_probe_router_info.json'))['sklearn_version'])"
python -c "import sklearn; print(sklearn.__version__)"
```

If the two numbers are the same, nothing more is needed. If they differ, the pipeline will stop at the start and say so. Either install the matching version (`pip install "scikit-learn==X.Y.Z"` with the first number) or train a new router as below. If you chose Path A (base models) in section 7, train a new one anyway after you have history.

11.3 Train or retrain the router

The router learns from your history, so it needs **at least 200 rows with Tier 1 quality marks, covering at least 5 question types**. A fresh copy of the project has no history, so first make some (this costs about $0.15 per 100 questions, mostly for marking):

```
python -m src.pipeline 300 --threshold-router
```

(1,000 questions is better.) Then train:

```
python scripts/train_text_router.py
```

You should see it evaluate itself (a table comparing "Tier 1 confidence" with "Learned router"), report the cut-point and the share it would escalate in each unit, and end with `Saved models/router/text_probe_router.joblib`. It takes seconds.

Option	Meaning	Default
`--target-share 0.25`	Escalate about this share of questions (between 0.02 and 0.98). A lower number is cheaper but keeps more small-model answers	0.30
`--history PATH`	Learn from a different history file	`results/logs/results_history.csv`
`--model-path PATH`	Save the model somewhere else	`models/router/text_probe_router.joblib`

Retrain when: you change a small model (the router learned that model's habits), you add a business unit, your history has grown a lot, or the version message appears. If it says the history is missing the `quality_draft_overall` column, run `python scripts/backfill_quality_scores.py` and then `python scripts/backfill_quality_scores.py --draft` (section 13).

11.4 What the router has learned

Open `models/router/what_it_learned.txt` in a text editor: it lists the words in questions and in answers that push the predicted quality down or up. They are associations in the data, not causes, but they are a good way to explain what the router pays attention to. `text_probe_router_info.json` records its version, cut-point and held-out results (`models/README.md` explains each field).

11.5 Comparing with the old rule

Run the same questions through both routers (set `SEED` to a number first, section 9):

```
python -m src.pipeline 100
python -m src.pipeline 100 --threshold-router
```

On the dashboard each run shows its router, and the run list tags the change. The pipeline never quietly falls back from one router to the other.

---

12. The dashboard

A web page that shows how the system is doing. It reads the log files by itself, so there is nothing to set up beyond installing its packages, which section 5 already did.

1. With `(venv)` showing, start it:

   ```
   python -m uvicorn dashboard.backend.main:app --port 8000
   ```

2. **You should see** a few lines ending with `Uvicorn running on http://127.0.0.1:8000`. **Leave this window open.**
3. In your web browser go to **http://localhost:8000**.

To stop it, click into that terminal and press `Ctrl + C`. To use it again later, repeat steps 1 to 3 (nothing needs reinstalling). If port 8000 is busy, add `--port 8010` and open that address instead. It refreshes every five seconds, so you can watch a run happen.

What you will see

- **Left: the list of runs**, newest first. Tags show `latest`, **new models** (the small models changed) and **new router** (the router was retrained or switched).
- **The summary** of a run: how many questions were answered locally, from the cache and by Claude; the router and its cut-point; confidence; quality; times; and what Claude cost.
- **Where each answer landed**: a dot per question. Under the learned router, each dot sits at the quality the router *predicted*, with the cut-point as a line (left of it is escalated). Under the older rule, dots sit at their confidence with the threshold lines.
- **Answer quality** by who produced the answer (the payments model, the retail-bank model, Claude, the cache), and the **latest questions** of the run.
- **Across runs**: trend charts with dashed markers where something changed, so you can compare before and after.

To compare runs fairly, use the same questions in each (set `SEED`, section 9) and give retrained models new names. `dashboard/README.md` has more, including how to change the page's appearance (this needs Node).

---

13. Quality scoring, the cache and the maintenance scripts

13.1 Quality scoring (on by default)

After each run, Claude (acting as an examiner, using a more careful model than the one that writes escalated answers) marks every question's **final answer**, and every escalated question's **discarded small-model answer**, from 1 to 5 on correctness, completeness, tone, safety and clarity. The average is `quality_overall`. Local answers copy their marks onto the "draft" columns for free, so no cell is left blank. This is what lets the project compare the small models' answers with Claude's fairly, and what the router learns from.

An honest limit: these are Claude's marks. They have not been checked against human markers.

13.2 The cache (on by default)

For a question the router escalated, the cache looks for an earlier Claude answer to a near-identical question. It must be very similar (at least 0.95), must not differ in negation ("cancel my card" against "don't cancel my card"), and must have the same category and intent. A hit is free and quick. The first time it is used it downloads a small model (about 90 MB), so the first run is slower to start. If that download fails, the run goes on without the cache and says so. Turn it off with `--nocache`.

13.3 The maintenance scripts

All are run from the project folder with `(venv)` showing. All are safe to run again: they do nothing if there is nothing to do.

Command	What it does
`python scripts/backfill_quality_scores.py`	Marks answers that have no quality mark yet (costs about $0.0012 per mark with Claude)
`python scripts/backfill_quality_scores.py --draft`	The same for the small model's own answers. Escalated rows are marked by Claude; local rows copy their marks. **Run the plain version first, then this one**
`... --unit payments` or `--unit retail_bank`	Only that business unit
`... --run "run-05-Oct-26-11:04 AM"`	Only that run
`... --limit 100`	At most 100 rows this time. Use it to check cost before doing everything
`... --judge-local MODEL`	Mark with a local Ollama model instead of Claude, into separate `quality_local_*` columns (a cheap first look, not the numbers to report)
`python scripts/run_and_score.py 100`	The pipeline, then both quality fills as a safety net. Any option after the number goes to the pipeline
`python scripts/recover_missing_run.py`	Adds a finished run that did not reach the history file because the file was open in Excel
`python scripts/generate_questions.py`	Makes a batch of fresh customer questions with a separate local model, for `--questions`. Needs `ollama pull qwen2.5:7b` first
`python scripts/evaluate_label_judge.py`	Measures how well a model can tell which request type a question expresses, for six ways of asking, using pool questions with known labels. Run it before trusting the generator's label check
`python scripts/redact_history.py`	Removes personal identifiers from logs already written. Reports first; `--apply` writes, after making a backup
`python`	Measures what the personal-data filter catches and misses on generated test cases

`scripts/evaluate_privacy_filter.py`	
`python scripts/backfill_cache_decision.py`	One-off: relabels old cache hits as `CACHE` in older logs
`python scripts/backfill_prompt_version.py`	One-off: labels older escalated rows with which Claude prompt wording was used
`python scripts/backfill_run_id.py`	One-off: gives logs from before run ids existed a run id. The pipeline tells you if you need it

---

14. Train your own models on Google Colab (optional)

This is the most advanced part. It teaches the two small models to be better at banking questions, and it is how the project's version 2 models were made. It is optional: the system works without it (Path A in section 7). It needs a powerful graphics card, so instead of your computer you borrow one from Google, using **Google Colab**.

What to expect. On a Colab **L4** graphics card, version 2 took about **3 hours 44 minutes** for the payments model and an estimated **5.5 hours** for the retail-bank model. On the free T4 card it is about twice as slow. Colab's paid tiers charge by the hour; read the current rate from Colab's own screen. Keep the Colab browser tab open and your computer awake while training. If a session drops, you can continue (see below).

Everything below is typed into Colab, not your own terminal. Run each block as a separate "cell" (click **+ Code**, paste, press the Play button or `Shift + Enter`).

1. **Open Colab with a GPU.** Go to **colab.research.google.com**, log in with your Google account, click **New notebook**, then **Runtime, Change runtime type**, choose **L4 GPU**, and **Save**.
2. **Connect Google Drive.** This saves your trained models safely, because Colab's own storage is wiped when a session ends:

   ```
   from google.colab import drive
   drive.mount('/content/drive')
   ```

   A pop-up asks you to log in and allow access. **You should see** `Mounted at /content/drive`.
3. **Download the project onto your Drive:**

   ```
   %cd /content/drive/MyDrive
   !git clone https://github.com/bipul-ranjan/two-tier-system.git
   %cd /content/drive/MyDrive/two-tier-system
   ```

   (If the folder is already there from before, run `!git pull` inside it instead.) For a private copy of the project, store a GitHub token as a Colab "Secret" (the key icon on the left, name `GITHUB_TOKEN`, notebook access on) and clone with it, so it never appears in a cell.
4. **Install the training software:**

   ```
   !pip install -r training/requirements.txt
   ```
5. **Download the training data:**

   ```
   !python scripts/fetch_bitext_banking.py
   ```
6. **Train the Payment Assistant.** This is the long one:

   ```
   !python training/train_payments.py
   ```

   A stream of numbers scrolls by; that is normal progress information. It saves a checkpoint to Drive every 250 steps, and finishes by saving the model to `LJMU_Research/two-tier-system-models` on your Drive.
7. **Restart before the second model.** Click **Runtime, Restart runtime** (this clears the first model from the graphics card's memory; skipping it can cause an out-of-memory error). A restart clears everything, so repeat steps 2 and 3 (and 4 and 5 if the installs were lost). Then make the extra training data for the retail model and train it:

```
!python scripts/generate_exception_training_data.py
!python training/train_retail.py
```

Check the line it prints, "Loaded ... existing pipeline-test instructions to avoid duplicating". The number must not be 0. That step builds training examples for fraud and hardship scenarios and removes any that also appear among the project's test questions, so the models are never tested on what they were trained on. A 0 would mean that protection is not working.

If the session drops partway: open a new session, repeat steps 2 to 5, and run the **same training command again**. It finds the latest checkpoint on Drive and continues ("Found an existing checkpoint ... resuming").

Always run these commands from the project folder (the folder holding `training`, `scripts` and `data`, which is where step 3 left you). They look for `data/raw/...` from there.

`training/README.md` has the settings used, timings, and a table of problems and fixes.

---

15. Bring your trained models into Ollama

The trained models are now files on your Google Drive. You need them on your own computer, then Ollama must be told about them.

15.1 Get the files onto your computer

Easiest: install **Google Drive for desktop** (google.com/drive/download), sign in with the same Google account, and choose **Mirror files** so a real copy is kept on your computer. Wait for the syncing icon to finish. The models are in `LJMU_Research/two-tier-system-models`. Alternatively, in Colab click the folder icon on the left, find the model files, right-click and choose Download.

Look for one `.gguf` file per model. Unsloth may put each in a folder named like `payment_assistant_gguf`, together with a small file called `Modelfile`.

15.2 Create the Ollama models

1. Open a terminal and move into the folder holding the **payments** `.gguf` file, for example (Windows):

   ```
   cd "G:\My Drive\LJMU_Research\two-tier-system-models\payment_assistant_gguf"
   ```

   (Mac: `cd "/Users/YOUR-NAME/Google Drive/My Drive/LJMU_Research/two-tier-system-models/payment_assistant_gguf"`.) Type `dir` / `ls` to see the `.gguf` file.

2. A file called **`Modelfile`** (no extension) must be in the same folder, containing one line, with the real file name of your `.gguf`:

   ```
   FROM ./payment_assistant.gguf
   ```

   If it is missing, create it in Notepad (Windows: set "Save as type" to **All Files**, so no `.txt` is added; Mac: use plain-text mode in TextEdit).

3. Create the model, using exactly this name (it is the name `src/config.py` expects):

   ```
   ollama create payment-assistant-v2 -f Modelfile
   ```

4. Do the same for the **retail-bank** file, in its own folder:

   ```
   ollama create retail-bank-assistant-v2 -f Modelfile
   ```

5. **Check:** `ollama list` shows `payment-assistant-v2` and `retail-bank-assistant-v2`. Try one:

   ```
   ollama run payment-assistant-v2 "Why was my card payment declined?"
   ```

15.3 Finish

- If you used Path A earlier, open `src/config.py` and put the model names back to `payment-assistant-v2` and `retail-

`bank-assistant-v2` (or leave your own names and be consistent).
- Check a real answer: `python -m src.tier1`.
- **Retrain the router** (section 11.3), because it learned the old models' habits. Give new models new names, so that the dashboard can mark the change.

---

16. Research scripts (optional)

These produced the evidence for choosing the router. You do not need them to run the system. They need history that has Tier 1 quality marks, and they use PyTorch (already installed).

Command	What it does and how long
`python scripts/probe_reward_model.py`	Tests whether a ready-made reward model predicts Claude's marks without training. About 25 minutes for 500 rows; downloads about 1.2 GB the first time. Saves as it goes, so `Ctrl + C` is safe and the same command continues. Options: `--n 200` (fewer rows), `--bf16` (half the memory), `--model ID`
`python scripts/train_router.py`	Compares MiniLM-based routers with confidence and the text probe, on question types held out of training. Frozen mode takes about 7 minutes
`python scripts/train_router.py --mode finetune --stop-after 1`	Fine-tunes MiniLM on one fold, about an hour on a processor, as a first look. Each fold is saved as it finishes, so a re-run continues
`python scripts/train_router.py --mode finetune --report-only`	Prints the report from folds already saved, training nothing

More options for `train_router.py`: `--folds`, `--epochs` (3), `--max-length` (256), `--batch-size` (16), `--lr` (5e-5), `--seed`, `--n` (a quick test on N rows), `--base-model`, `--history`, `--out-dir`. The results the project got are in `results/analysis/` and are summarised in the project documents: neither MiniLM router beat the simple text router, so the simple one is used.

---

17. Everyday commands

Start every session: open a terminal, go to the project folder, and activate the environment.

```
cd Desktop\two-tier-system            (Mac: cd Desktop/two-tier-system)
.\venv\Scripts\Activate.ps1           (Mac: source venv/bin/activate)
```

To do this	Type this
Run the system on 10 / 100 questions	`python -m src.pipeline 10` / `python -m src.pipeline 100`
Run without the quality marking (cheaper)	`python -m src.pipeline 100 --noquality`
Run without the cache	`python -m src.pipeline 100 --nocache`
Run with the older confidence router	`python -m src.pipeline 100 --threshold-router`
Run, then make sure every mark is filled in	`python scripts/run_and_score.py 100`
Train or retrain the router	`python scripts/train_text_router.py` (add `--target-share 0.25` to escalate fewer)
Fill in missing quality marks	`python scripts/backfill_quality_scores.py` then `python scripts/backfill_quality_scores.py --draft`
Summary tables for the latest run	`python -m src.evaluate`
Open the dashboard	`python -m uvicorn dashboard.backend.main:app --port 8000`, then open `http://localhost:8000`
See which models the project will use	`python -m src.config`
One real local answer / one real Claude answer	`python -m src.tier1` / `python -m src.tier2_escalate`
Run the tests	`python -m pytest tests/ -q`
See which models Ollama has / has loaded	`ollama list` / `ollama ps`
Talk to a model directly	`ollama run payment-assistant-v2 "your question"`
Rescue a run that was not saved to the history	`python scripts/recover_missing_run.py`

Save your changes to GitHub	`git add -A`, then `git commit -m "describe what changed"`, then `git push`
Get someone else's changes	`git pull`
Leave the environment	`deactivate`

---

18. Tests

```
python -m pytest tests/ -q
```

You should see `158 passed` in about two seconds. They need no Ollama, no Claude key and no data. Twelve test the learned router (its predictions, its exact cut-point rule, saving and loading, and each error message) and four test the older confidence rule. Run them after changing the router, after upgrading scikit-learn, and before sharing a copy. `tests/README.md` lists each test and what is not covered.

---

19. If something goes wrong

If you meet an error not listed here, copy the exact error message somewhere safe before doing anything else: it usually says exactly what went wrong.

What you see	What it means, and what to do
`'ollama' is not recognized` or `command not found: ollama`	Ollama is not installed, or the terminal was opened before it was. Open a **new** terminal
`No module named ...` (any module)	The environment is not active. Look for `(venv)` at the start of the line; if it is missing, repeat section 5, step 2. If it is there, run `pip install -r requirements.txt` again
"The learned router has not been trained yet"	`models/router/text_probe_router.joblib` is missing. Train it (section 11.3) or run once with `--threshold-router`
"was trained with scikit-learn X, but Y is installed"	Install the matching version, or retrain (section 11.2 and 11.3)
"only has N rows ... need at least 200" when training the router	You have too little history. Run `python -m src.pipeline 300 --threshold-router`, then train
"is missing columns ['quality_draft_overall']"	Run `python scripts/backfill_quality_scores.py` then `python scripts/backfill_quality_scores.py --draft`
`model "payment-assistant-v2" not found`	The fine-tuned model is not in Ollama. Do section 15, or switch `src/config.py` to the base models (section 7, Path A)
The program ignores the new model names	You edited a different copy of `src/config.py`, or saved it as `config.py.txt`. Check the exact file name
Claude questions fail with an authorization error	The key is not set in this window. Section 6: on Windows, close the terminal, open a new one and activate the environment again
`PermissionError` on `results_history.csv`, or "locked"	Another program (Excel, an editor, another run) has it open. Close it. If a run finished but is missing from the history, run `python scripts/recover_missing_run.py`
"... has no run_id column and this run would overwrite it"	Older logs from before run ids existed. Run `python scripts/backfill_run_id.py` once
A warning that the models are not both "resident"	The two models do not fit in memory together, so Ollama swaps them and every answer is slow. Close other programs, or use smaller models
Answers take more than 30 seconds, or a request times out	Ollama is busy or short of memory. Close other programs. Each call waits up to 120 seconds and tries 3 times before skipping the question
"semantic cache unavailable"	The embedding model could not download, or `sentence-transformers` is missing. The run continues without the cache. Check your internet connection, or use `--nocache`
`Connection refused` when opening the dashboard	The dashboard is not running. Start it (section 12) and keep that window open
`Address already in use`	Port 8000 is taken. Add `--port 8010` and open that address
The dashboard shows old numbers or old layout	Hard-refresh the browser with `Ctrl + F5`
Git asks for a password and rejects it	Use a personal access token instead (section 4.2)
Colab training stops partway	The tab closed, the computer slept, or the session timed out. Start a new session and repeat steps 2 to 5 of section 14, then run the **same** training command: it resumes from the last checkpoint

Colab: "Google Drive is not mounted"	Run the mount cell first (section 14, step 2)
Colab: `data/raw/... not found`	You are not in the project folder, or the data step was skipped (section 14, steps 3 and 5)
Colab: out of memory on the second model	Click Runtime, Restart runtime, then repeat the set-up steps before training the retail model
A file you edited did not seem to save	Some editors add a hidden `.txt` to file names. Check the exact name, and save with "All Files" as the type

---

20. Appendices

A. The folders

Folder or file	What is in it
`README.md`	The overview, installation and all commands in short
`requirements.txt`	The Python packages to install (section 5)
`src/`	The pipeline: Tier 1, router, cache, Tier 2, quality scoring, logging (`src/README.md`)
`scripts/`	Data, maintenance, router-training and research scripts (`scripts/README.md`)
`training/`	Fine-tuning scripts for Colab (`training/README.md`)
`models/router/`	The trained router and its list of "words it weighs" (`models/README.md`)
`prompts/`	Extra instructions added to the small models' prompts (`prompts/README.md`)
`data/synthetic/`	The ten files of practice questions (included)
`data/raw/`	Downloaded datasets (created by scripts; not included) (`data/README.md`)
`dashboard/`	The dashboard (`dashboard/README.md`)
`tests/`	The 158 automated tests (`tests/README.md`)
`results/logs/`	Every run's records, created by the pipeline (`results/logs/README.md`)
`results/tables/`, `results/plots/`, `results/analysis/`	Summary tables, charts, and the saved research outputs
`User-Manual/`	This manual

B. The settings you can change, and where

Setting	Where	What it does
Which model each unit uses	`src/config.py`	Names the Ollama model and the persona for each business unit
Number of questions, repeatable questions	`src/pipeline.py` (`N_SAMPLES`, `SEED`)	Default run size; `SEED = None` is random, a number repeats
The older router's pass marks	`src/pipeline.py` (`THRESHOLDS`)	0.71 payments, 0.60 retail bank
How many questions the router escalates	`python scripts/train_text_router.py --target-share`	Sets the cut-point
How alike a cached question must be	`src/semantic_cache.py` (`SIMILARITY_THRESHOLD`)	0.95
Extra instructions for the small models	`prompts/business_unit_prompts.json`	One block per business unit
The Claude escalation model and its prices	`src/tier2_escalate.py`	Check Anthropic's price list before relying on cost figures
The marking rubric	`src/quality.py` (`_QUALITY_RUBRIC`)	One place; changes every path that scores quality
Where the dashboard reads logs	environment variable `TWO_TIER_LOGS_DIR`	Default `results/logs`
Your Claude key	environment variable `ANTHROPIC_API_KEY`	Never put it in a file

C. Versions, and how to record yours

The project was built and tested with Python 3.12 and 3.14, and the author's computer runs Windows with PowerShell. Package versions are not fixed in `requirements.txt`, so a newer package may behave differently. The one that matters most is scikit-learn, because a saved router only loads under the version that trained it. To record exactly what works on your computer, with the environment active:

```
pip freeze > requirements-lock.txt
```

and keep that file. To rebuild the same set later: `pip install -r requirements-lock.txt`.

D. What the system cannot tell you

Please keep these limits in mind whenever you quote a result. All questions are synthetic. The quality marks are Claude's and have not been checked against people. The system has not been tested against attacks such as prompt injection, and is not ready for real customer data. On the author's computer a local answer takes about as long as a Claude call, so the benefits shown are cost and keeping most answers on the computer, not speed. Several tests used only 10 to 500 questions. The router is tied to the models it was trained on, so retrain it when you change them.